

Data Visualization en Python

Arquitectura, historia y ecosistema del análisis visual

Valentino Ortiz · Laureano Oliva · Facundo Ramirez
UTN FRRo · Seminario técnico · 2026



Objetivos del seminario

01

Comprender la arquitectura interna

Capas, objetos y flujo de renderizado de Matplotlib, Seaborn y Plotly.

02

Diferenciar paradigmas

Procedimental, orientado a objetos, agregación estadística y serialización a JSON.

03

Elegir con criterio técnico

Qué herramienta usar según la restricción: PDF, exploración rápida o tablero interactivo.

De Tufte a Matplotlib: una breve historia

1983

2000's

2003

Hoy



Edward Tufte publica *The Visual Display of Quantitative Information*: gráficos honestos, limpios y sin adornos.



MATLAB es el estándar para graficar: producto comercial y de código cerrado.



John D. Hunter, neurobiólogo, crea Matplotlib en Python imitando los comandos de MATLAB.



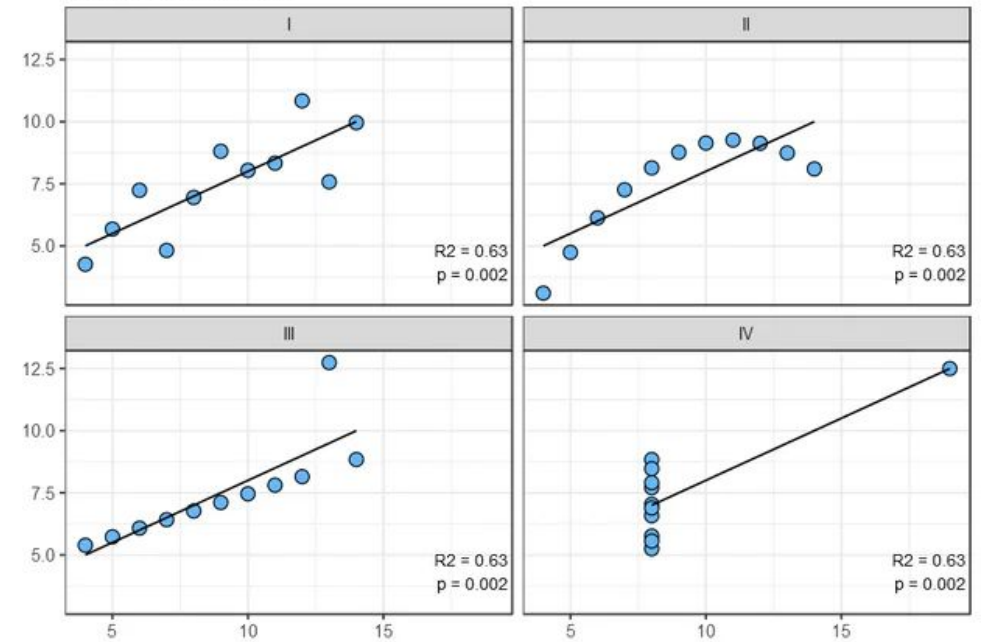
Pandas, Seaborn y gran parte del ecosistema científico de Python dibujan sobre Matplotlib.

El Cuarteto de Anscombe (1973)

Cuarteto de Anscombe

I		II		III		IV	
x	y	x	y	x	y	x	y
10.0	8.04	10.0	9.14	10.0	7.46	8.0	6.58
8.0	6.95	8.0	8.14	8.0	6.77	8.0	5.76
13.0	7.58	13.0	8.74	13.0	12.74	8.0	7.71
9.0	8.81	9.0	8.77	9.0	7.11	8.0	8.84
11.0	8.33	11.0	9.26	11.0	7.81	8.0	8.47
14.0	9.96	14.0	8.10	14.0	8.84	8.0	7.04
6.0	7.24	6.0	6.13	6.0	6.08	8.0	5.25
4.0	4.26	4.0	3.10	4.0	5.39	19.0	12.50
12.0	10.84	12.0	9.13	12.0	8.15	8.0	5.56
7.0	4.82	7.0	7.26	7.0	6.42	8.0	7.91
5.0	5.68	5.0	4.74	5.0	5.73	8.0	6.89

Anscombe's Quartet



Versión moderna: el Datasaurus Dozen

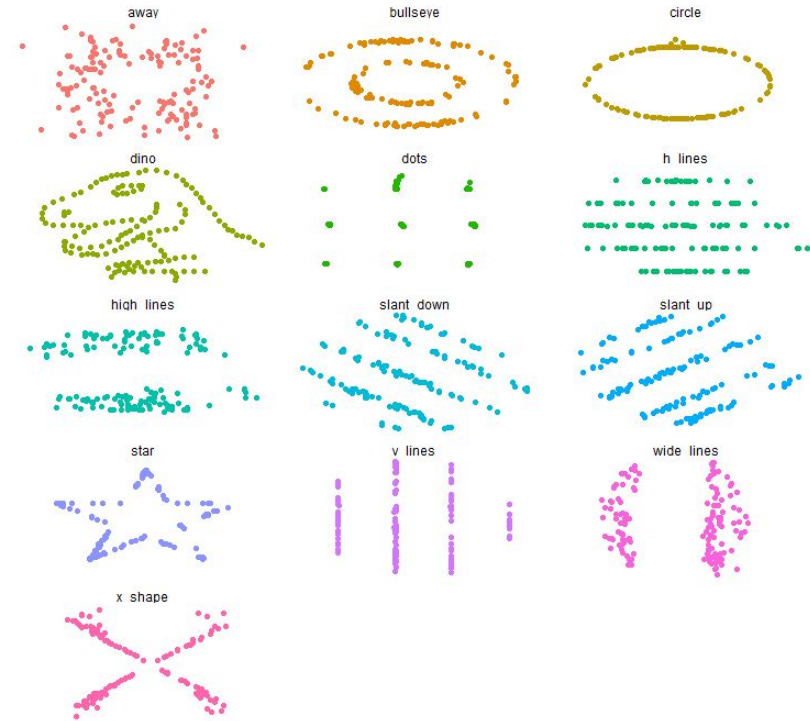
2016

Alberto Cairo crea el Datasaurus: un conjunto de puntos que dibuja un dinosaurio.

2017

Matejka y Fitzmaurice (Autodesk Research) generan otros 12 conjuntos con formas distintas: estrella, círculo, X.

Misma media, misma desviación estándar y misma correlación, hasta el segundo decimal.



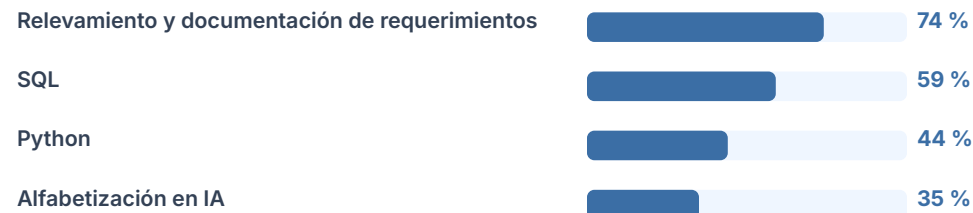
Mismos números, dibujos completamente distintos.

Demanda laboral en data analytics: Argentina

Demanda destacada

- Ciencia de datos y analítica lideran la demanda y ofrecen las mejores remuneraciones del sector tecnológico argentino.
- **Randstad, carreras con mayor proyección 2026:** se buscan especialistas en Business Intelligence y analítica avanzada en empresas de todos los tamaños.

Competencias más buscadas



Las más pedidas son las fundacionales, no las más novedosas.

Perfiles con más tracción

- **Data Engineer:** construye y mantiene los pipelines de datos que alimentan los análisis.
- **Data Scientist / AI/ML Engineer:** analítica predictiva y automatización de decisiones. La demanda crece a doble dígito.

Quién contrata

- **Multinacionales:** L'Oréal, Novartis, PedidosYa y Naranja X.
- **GlobalLogic** planea incorporar más de 450 profesionales en Argentina durante 2026.
- **Ofertas 100 % remotas** desde Argentina: EPAM, Webflow y Newsela.

La tríada de visualización en Python

Matplotlib

La librería madre

Motor de bajo nivel. Control total sobre cada elemento, a cambio de más código.

Seaborn

Estadística de alto nivel

Construida sobre Matplotlib. Exploración rápida con DataFrames de Pandas.

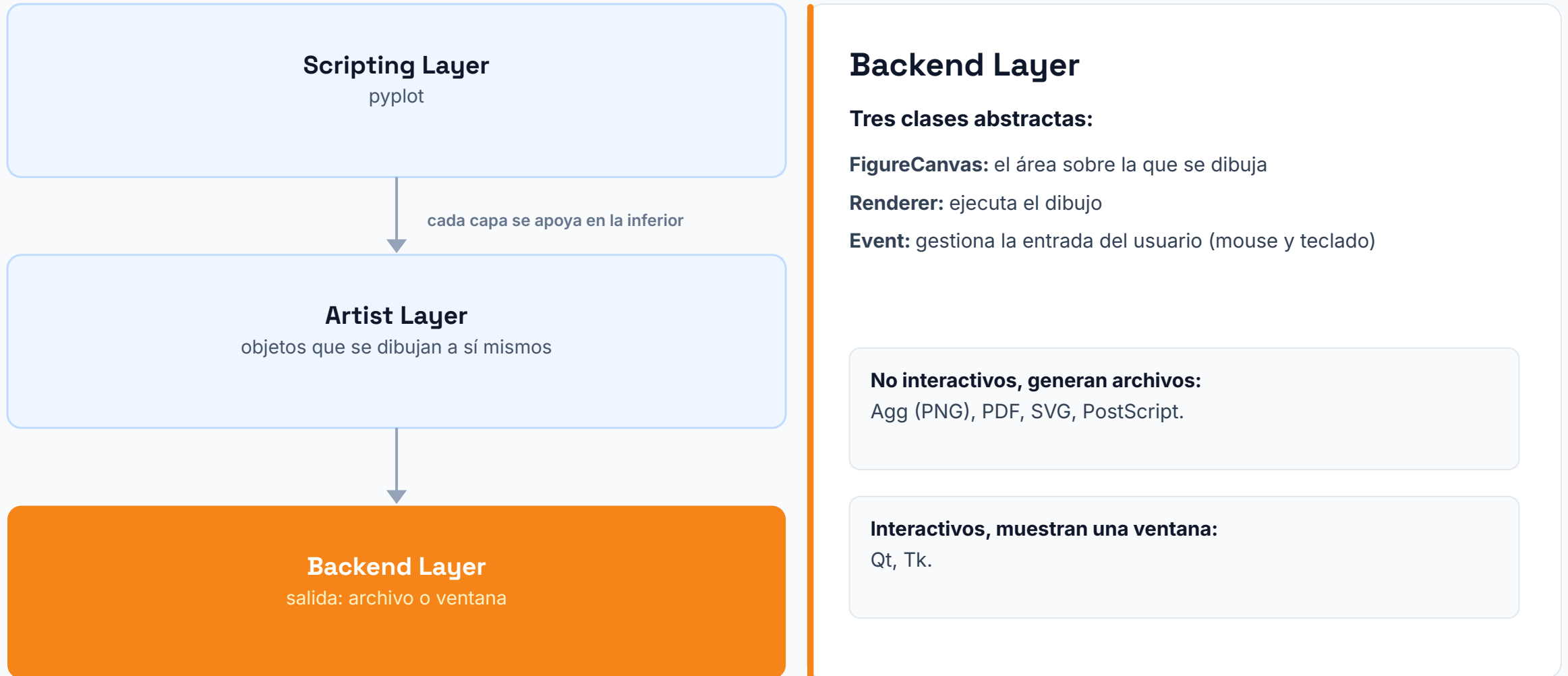
Plotly

Interactivo y web

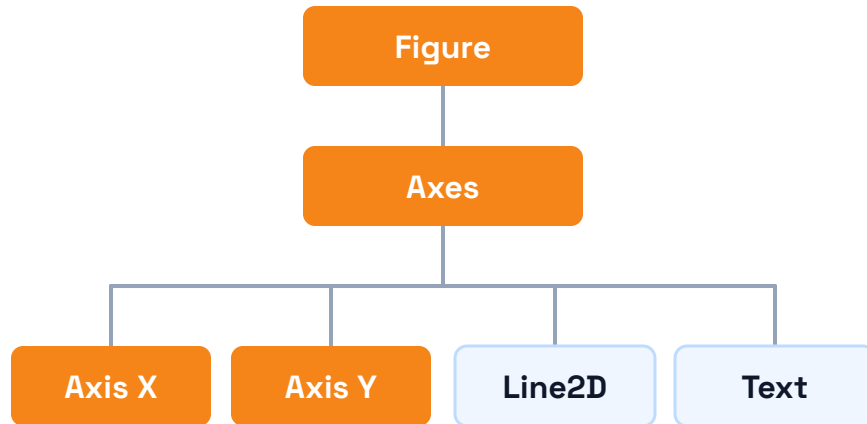
Estándar para gráficos interactivos y dashboards en el navegador.

Para elegir entre ellas, hay que entender qué pasa por dentro de cada una.

Matplotlib: arquitectura en tres capas



Artist Layer: todo lo que se ve es un Artist



= contenedor = primitiva

Primitivas: Line2D, Rectangle, Text.

Contenedores: Figure, Axes, Axis. Agrupan a otros artistas.

Cada Artist conoce sus coordenadas y se dibuja a sí mismo cuando recibe un Renderer.

Axes = área de dibujo con su sistema de coordenadas. **Axis** = eje X o eje Y.

Scripting Layer: pyplot y la interfaz orientada a objetos

pyplot (estado global)

- Imita a MATLAB.
- Recuerda la figura y los ejes actuales.
- Cómodo en scripts cortos; ambiguo en código grande.

```
# Estilo procedimental / estado global
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [1, 4, 9])
plt.title("pyplot")
plt.show()
```

Orientado a objetos (explícito)

- plt.subplots() crea la Figure y los Axes.
- Se trabaja con métodos de cada ax.
- Recomendado en software mantenible.

```
# Estilo explícito POO
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([1, 2, 3], [1, 4, 9])
ax.set_title("Orientado a objetos")
plt.show()
```

Resumen: como un taller de impresión. pyplot es el pedido al asistente, Artist es el diseñador y el backend es la imprenta.

Matplotlib: tipos de gráficos y sintaxis

Familia	Función	Sintaxis básica
Líneas	<code>ax.plot</code>	<code>ax.plot(x, y, label="serie", color="tab:blue")</code>
Dispersión	<code>ax.scatter</code>	<code>ax.scatter(x, y, s=tamaños, c=colores)</code>
Barras	<code>ax.bar</code> / <code>ax.barh</code>	<code>ax.bar(categorias, alturas)</code>
Histograma	<code>ax.hist</code>	<code>ax.hist(datos, bins=20)</code>
Cajas	<code>ax.boxplot</code>	<code>ax.boxplot(datos)</code>
Imágenes / mapas de calor	<code>ax.imshow</code>	<code>ax.imshow(matriz, cmap="viridis")</code>
Personalización	<code>ax.set_title</code> , <code>set_xlabel</code> , <code>set_ylabel</code> , <code>set_xlim</code> , <code>legend</code> , <code>grid</code> , <code>annotate</code>	<code>ax.set_title("Título")</code>
Disposición y salida	<code>plt.subplots</code> , <code>fig.tight_layout</code> , <code>fig.savefig</code>	<code>fig.savefig("grafico.pdf")</code>

Patrón: `ax.tipo_de_gráfico(datos, parámetros)` y luego `ax.set_*`() para etiquetar.

```

import numpy as np
import matplotlib.pyplot as plt
axs[0, 0].plot(x, np.sin(x), label="sin(x)")
axs[0, 0].plot(x, np.cos(x), label="cos(x)")
axs[0, 0].set_title("plot: líneas")
axs[0, 0].set_xlabel("x")
axs[0, 0].legend()

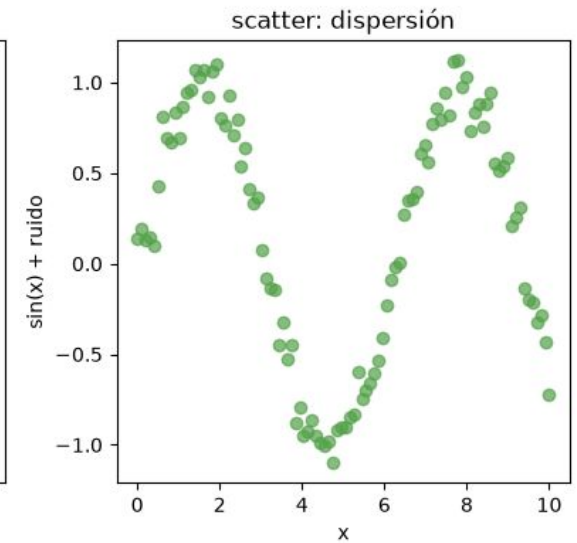
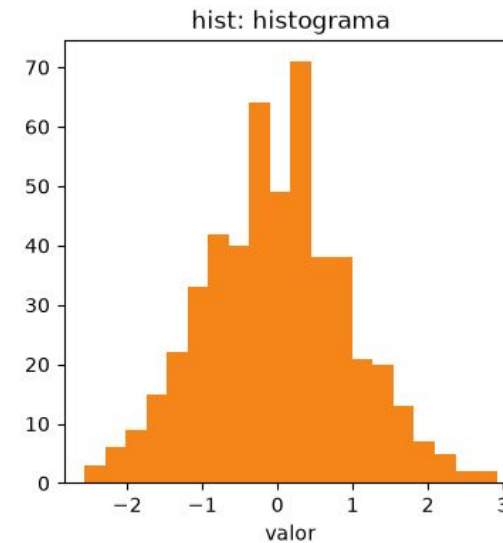
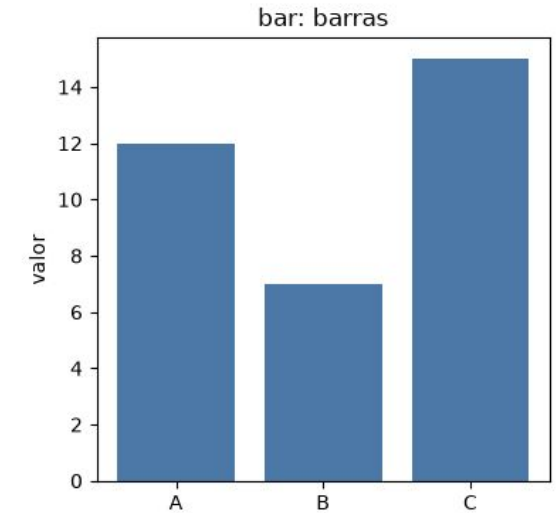
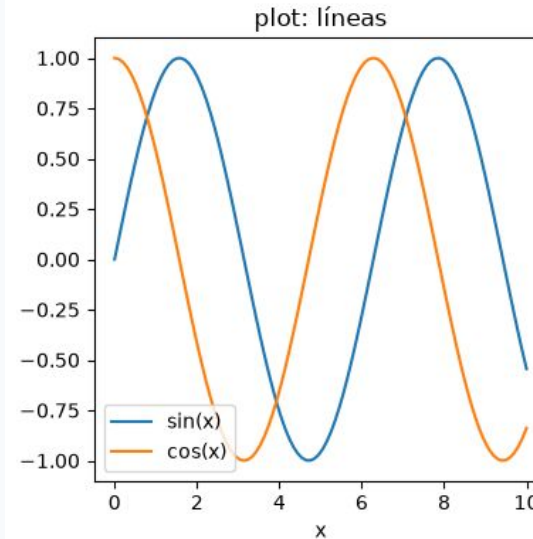
axs[0, 1].bar(["A", "B", "C"], [12, 7, 15], color="#4C78A8")
axs[0, 1].set_title("bar: barras")
axs[0, 1].set_ylabel("valor")

axs[1, 0].hist(rng.normal(0, 1, 500), bins=20, color="#F58518")
axs[1, 0].set_title("hist: histograma")
axs[1, 0].set_xlabel("valor")

axs[1, 1].scatter(x, np.sin(x) + rng.normal(0, 0.1, 100), color="#54A24B",
alpha=0.7)
axs[1, 1].set_title("scatter: dispersión")
axs[1, 1].set_xlabel("x")
axs[1, 1].set_ylabel("sin(x) + ruido")

fig.tight_layout()

```



Seaborn: estadística de alto nivel sobre Matplotlib

Cómo se usa

Entrada: un DataFrame de Pandas en formato tidy (una fila por observación, una columna por variable).

```
data = el DataFrame
x, y = columnas de cada eje
hue = columna que define el color
```

Familias de funciones

- **Relacionales:** scatterplot, lineplot
- **Distribución:** histplot, kdeplot
- **Catóricas:** boxplot, barplot, countplot
- **Regresión:** regplot, lmlplot
- **Matrices:** heatmap
- **Grillas:** pairplot

1

Agrega: agrupa y calcula la media

2

Estima la incertidumbre:
bootstrapping, IC 95 %

3

Aplica una paleta de colores

4

Dibuja con objetos de Matplotlib.

Resumen: Seaborn automatiza la agregación estadística y la estimación visualizando DataFrames de forma directa sobre Matplotlib.

Matplotlib

```
promedio = tips.groupby("day", observed=True)["tip"].mean()
```

```
fig, ax = plt.subplots()
```

```
barras = ax.bar(promedio.index.astype(str), promedio.values,  
width=0.6)
```

```
ax.bar_label(barras, fmt="%.2f", padding=3)
```

```
ax.set(title="Propina promedio por día", ylabel="Propina  
promedio (USD)")
```

```
ax.grid(axis="x", visible=False)
```

```
plt.show()
```

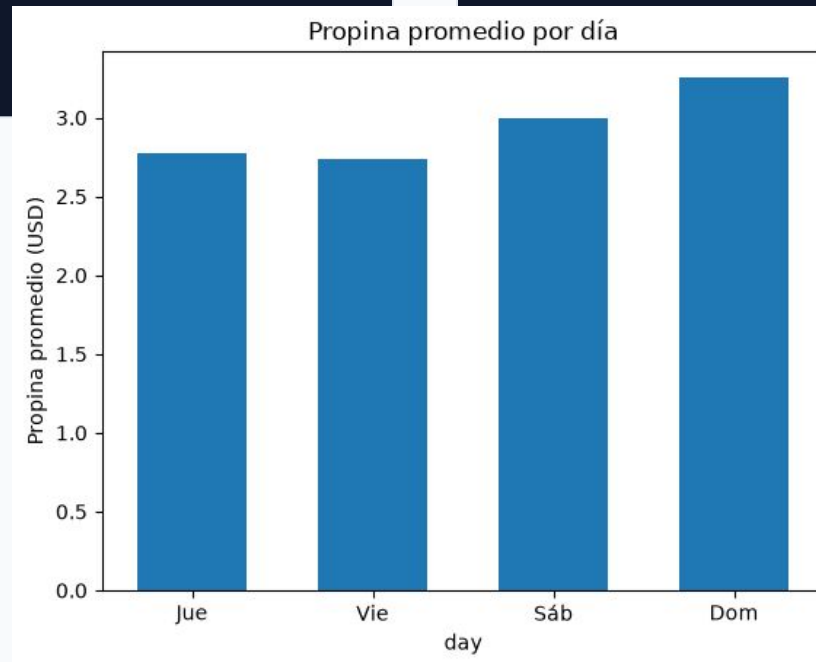
Seaborn

```
fig, ax = plt.subplots()
```

```
sns.barplot(data=tips, x="day", y="tip", estimator="mean",  
errorbar=None, saturation=1, width=0.6, ax=ax)
```

```
ax.set(title="Propina promedio por día", ylabel="Propina  
promedio (USD)")
```

```
plt.show()
```



Plotly: la figura es una especificación JSON

plotly.express (alto nivel)

Una llamada crea un gráfico completo.

```
px.scatter  
px.line  
px.bar  
px.histogram  
px.box
```

plotly.graph_objects (bajo nivel)

Se construye trazo por trazo, con control total.

```
go.Figure  
go.Scatter
```

La Figure

Es un diccionario con dos claves:

```
data = lista de trazas  
layout = títulos, ejes, márgenes
```

Python (Figure)



JSON



Navegador o Jupyter (Plotly.js)

Renderiza en SVG; usa WebGL con muchos puntos.

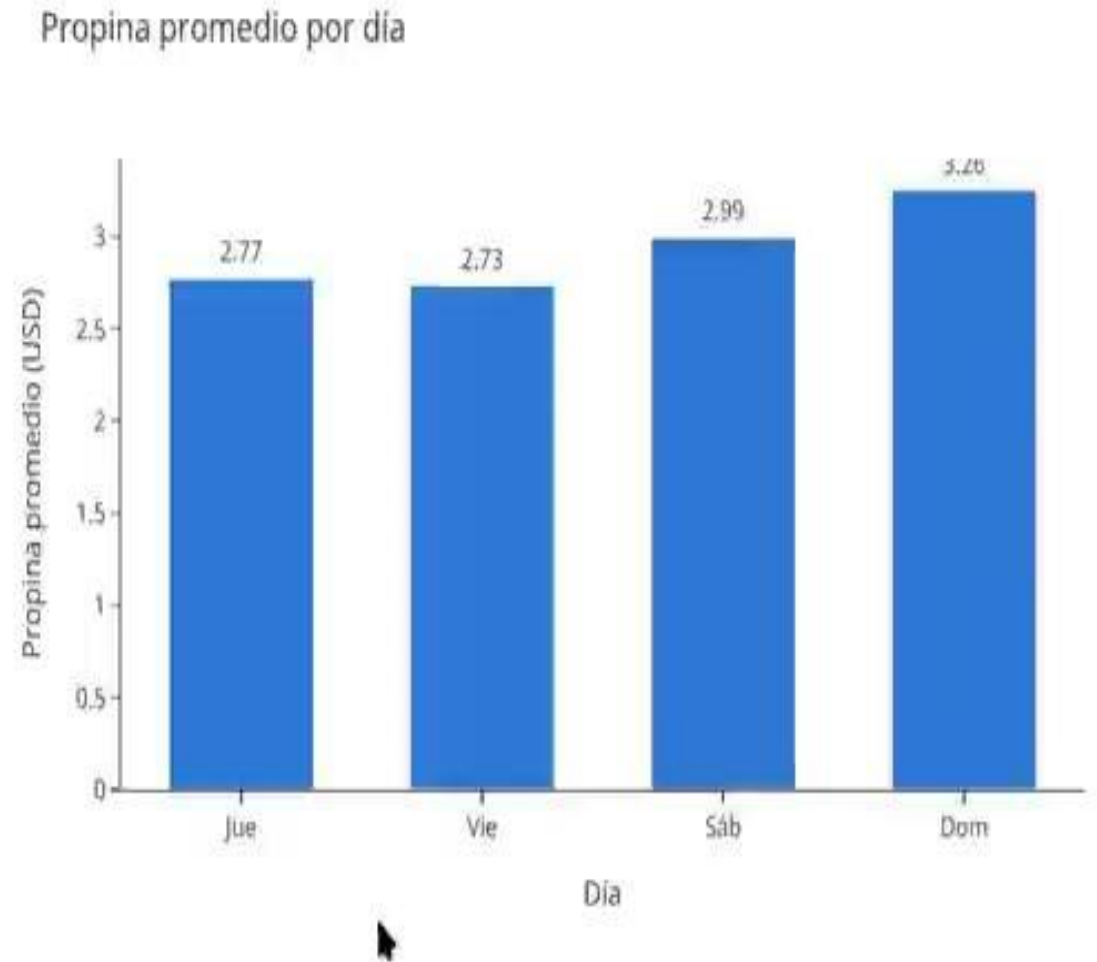
Zoom, tooltips y filtro por leyenda los procesa el navegador, no el kernel de Python.

Plotly: un gráfico interactivo en una llamada

```
import plotly.express as px

fig = px.bar(promedio.reset_index(), x="day",
             y="tip", text_auto=".2f",
             category_orders={"day": ORDEN_DIAS},
             color_discrete_sequence=[AZUL],
             labels={**ETIQUETAS, "tip": "Propina
promedio (USD)"},
             title="Propina promedio por día",
             template="simple_white", width=700, height=450)

fig.update_traces(width=0.6, textposition="outside")
fig.show()
```



Principios de diseño y accesibilidad

1

Maximizar el data-ink ratio (Tufte)

La mayor parte de la tinta debe representar datos: fuera bordes pesados, fondos oscuros y grillas recargadas.

2

Evitar el chartjunk

Se desaconsejan los gráficos 3D de torta o de barras: la perspectiva distorsiona la percepción del área.

3

Color con intención

El color codifica datos: paletas categóricas para variables cualitativas; secuenciales o divergentes para cuantitativas.

4

Accesibilidad

Contraste adecuado (criterios WCAG) y paletas perceptualmente uniformes como Viridis o Cividis, legibles con daltonismo.

Comparativa: ventajas, desventajas y cuándo usar cada una

Herramienta	Ventajas	Desventajas	Cuándo usarla
Matplotlib	Control total; salida vectorial (PDF/SVG); base del ecosistema	Verboso; curva de aprendizaje alta; estética por defecto básica	Publicaciones y reportes estáticos
Seaborn	Rápido; estadística integrada; buena estética	Menos control fino; depende de Matplotlib	EDA (exploración inicial)
Plotly	Interactivo; despliegue web; integra con Dash y Streamlit	HTML pesado con muchos datos; menos control para impresión	Dashboards y apps
Otras	Altair y Bokeh (declarativo / interactivo), ggplot2 (R), Power BI y Tableau (BI sin código)	Menos flexibles, otro ecosistema o licencias	Según equipo y contexto

No hay una mejor herramienta: hay una mejor herramienta para cada restricción.

Conclusión: qué usar y para qué

Situación	Herramienta	Por qué
Explorar un dataset nuevo: correlaciones, outliers, distribuciones	Seaborn	Estadística integrada, una línea por gráfico
Figura para un paper, una tesis o un informe en PDF; curva de entrenamiento de un modelo	Matplotlib	Control total y salida vectorial
Tablero interactivo para gerencia o publicado en la web	Plotly (Dash / Streamlit)	Interactividad en el navegador
Auditar datos faltantes o mapear datos geográficos	Missingno / Folium ...	Herramientas de nicho

01

No hay una mejor librería: hay una mejor para cada restricción.

02

Flujo híbrido: explorar con Seaborn, refinar con Matplotlib, publicar con Plotly.

03

Grafiquen siempre los datos antes de concluir (Anscombe, Datasaurus).



¡Muchas gracias!

